Experiment No 1

AIM: Cryptography in Blockchain, Merkle root Tree Hash

Theory:

1) Cryptographic Hash Functions in Blockchain:

A cryptographic hash function converts input data of any size into a fixed-length string called a hash.

Key Properties:

- Deterministic – Same input always produces the same hash
- Fast computation – Hashes are generated quickly
- Pre-image resistance – Original data cannot be derived from the hash
- Collision resistance – Two different inputs cannot produce the same hash
- Avalanche effect – Small input change causes a large hash change

Role in Blockchain:

- Secures block data
- Links blocks together
- Enables Proof-of-Work mining
- Ensures data integrity and immutability

2) What is a Merkle Tree?

A Merkle Tree (also called a Hash Tree) is a data structure that organizes transaction data using cryptographic hashes.

- Leaf nodes contain hashes of individual transactions
- Parent nodes contain hashes of their child nodes
- The top hash is called the Merkle Root

The Merkle Root represents all transactions in a block.

What is a Cryptographic Puzzle & the Golden Nonce?

A cryptographic puzzle is a challenge miners must solve to add a new block to the blockchain.

Puzzle Goal:

Find a nonce value such that:

Hash(Block Data + Nonce) < Target

(or starts with a certain number of leading zeros)

Golden Nonce:

- The Golden Nonce is the first nonce that produces a valid hash meeting the difficulty requirement
- It proves computational work has been done
- Once found, the block is accepted by the network

This process is known as Proof-of-Work (PoW).

3) How Does a Merkle Tree Work?

1. Each transaction is hashed
2. Hashes are paired and concatenated
3. Each pair is hashed again to form parent nodes
4. If a level has an odd number of hashes, the last hash is duplicated
5. The process repeats until one hash remains → Merkle Root

Any change in a transaction changes the Merkle Root

4) Benefits of Merkle Tree

- Efficient verification of transactions
- Reduced storage requirements
- High security through hashing
- Quick detection of tampering
- Supports lightweight (SPV) clients

5) Use Cases of Merkle Tree

- Blockchain transaction verification
- Bitcoin & Ethereum block structure
- Data integrity verification
- Distributed file systems
- Git version control system
- Peer-to-peer networks
- Database synchronization

Tasks Performed:

1. **Hash Generation using SHA-256:**
   - Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

```
import hashlib

def generate_sha256_hash(input_string):

    encoded_string = input_string.encode('utf-8')

    hash_object = hashlib.sha256(encoded_string)

    return hash_object.hexdigest()

# Example usage
text = input("Enter a string: ")
print("SHA-256 Hash:", generate_sha256_hash(text))
```

Output:

```
Enter a string: Vedang
SHA-256 Hash: 31ef00ea9ae265c4681883ed25f84690ff34e160b1a032eb448c3055997ec446
```

2. **Target Hash Generation with Nonce:**
   - Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

   Code:

```
import hashlib

def hash_with_nonce(input_string, nonce):
    combined = f"{input_string}{nonce}"
    return hashlib.sha256(combined.encode('utf-8')).hexdigest()

# Example usage
data = input("Enter data: ")
nonce = int(input("Enter nonce: "))
```

```
print("Generated Hash:", hash_with_nonce(data, nonce))
```

Output:

```
Enter data: Vedang
Enter nonce: 2
Generated Hash: d06c523fbc36d280af6ca8ad1d2d525443007c66b05a9a336c6086be35a4f9fa
```

3. **Proof-of-Work Puzzle Solving:**
   - Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

Code:

```
import hashlib

def proof_of_work(input_string, difficulty):
    nonce = 0
    target = '0' * difficulty

    while True:
        combined = f"{input_string}{nonce}"
                                                hash_result      =
   hashlib.sha256(combined.encode('utf-8')).hexdigest()

        if hash_result.startswith(target):
            return nonce, hash_result

        nonce += 1

# Example usage
data = input("Enter data: ")
difficulty  =  int(input("Enter  difficulty  (number  of  leading
   zeros): "))

nonce, hash_result = proof_of_work(data, difficulty)
print("Nonce found:", nonce)
print("Hash:", hash_result)
```

Output:

```
Enter data: Wajge
Enter difficulty (number of leading zeros): 2
Nonce found: 44
Hash: 0008ef5674b632a97ad0eb0093fb1ed03d952b9eb5586638648b2abccc7440be
```

4. **Merkle Tree Construction:**
   - Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Code:

```python
import hashlib

def sha256(data):
    return hashlib.sha256(data.encode('utf-8')).hexdigest()

def merkle_root(transactions):

    hashes = [sha256(tx) for tx in transactions]

    while len(hashes) > 1:
        if len(hashes) % 2 != 0:
            hashes.append(hashes[-1])
        new_level = []
        for i in range(0, len(hashes), 2):
            combined_hash = sha256(hashes[i] + hashes[i + 1])
            new_level.append(combined_hash)

        hashes = new_level

    return hashes[0]

# Example usage
transactions = [
    "Vedang pays Diksha 10 BTC",
    "Aditi pays Siddhant 5 BTC",
    "Rutuja pays Shruti 2 BTC",
    "Dave pays Eve 1 BTC"
]
```

```
print("Merkle Root:", merkle_root(transactions))
```

Output:

```
Merkle Root: 9974430a5836b3816d28c8fefca2c324e2ca009693b1b590e43eafa56b89b14d
```

Conclusion:

This study highlights the basic cryptographic principles behind blockchain technology. SHA-256 hashing ensures data integrity by generating unique hashes, while nonce and target hashing demonstrate the proof-of-work process that makes block mining computationally difficult. Merkle Trees efficiently verify large numbers of transactions using a single root hash. Together, these techniques show how cryptography secures blockchain systems by ensuring data integrity, immutability, and efficient decentralized verification.